Curso
Tendencias Aplicadas de Ingenería de Software

Contenerización

Contenerización para Pruebas y Despliegue automatizado

- Analizar los siguientes temas:
  - Orquestación de despliegues
  - Docker vs Kubernetes
  - On-premises vs Cloud

Antes:

- Deploy manual
  - Ambientes inconsistentes
  - “Funciona en mi máquina”

Ahora:

- CI/CD
  - Infraestructura reproducible
  - Escalabilidad automática

Problema Clásico

Problema clásico

- Diferencias entre ambientes:
  - Dev
  - QA
  - Staging
  - Producción
- Problemas:
  - Dependencias distintas
  - Versiones incompatibles
  - Configuración manual

Solución más actualizada

- Contenerización:
  - Encapsular aplicación + dependencias
  - Entornos reproducibles
  - Portabilidad
- ¿Qué es un contenedor?

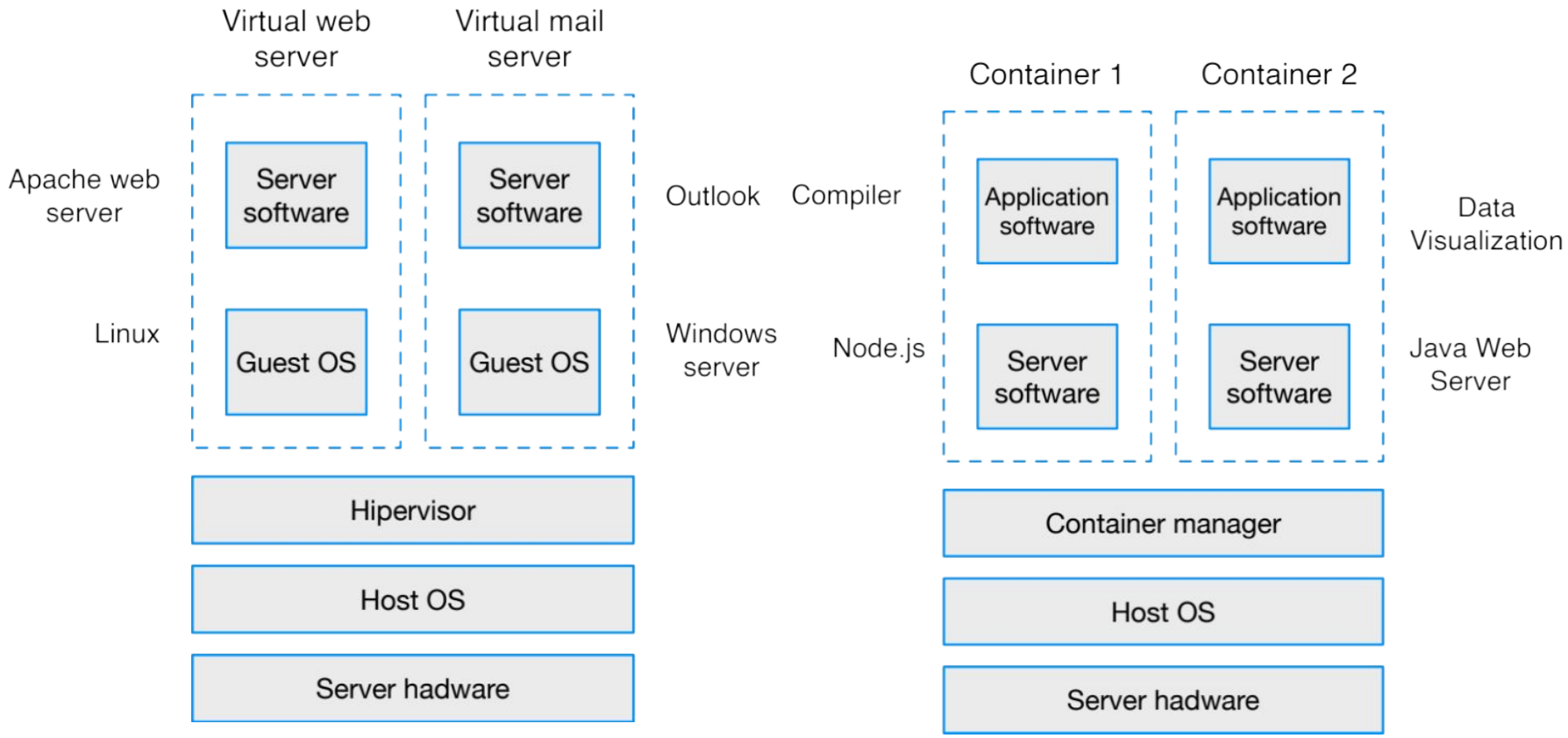
  Un contenedor es: Una unidad ligera y ejecutable que empaqueta una aplicación con sus dependencias.

- **No es una VM.**

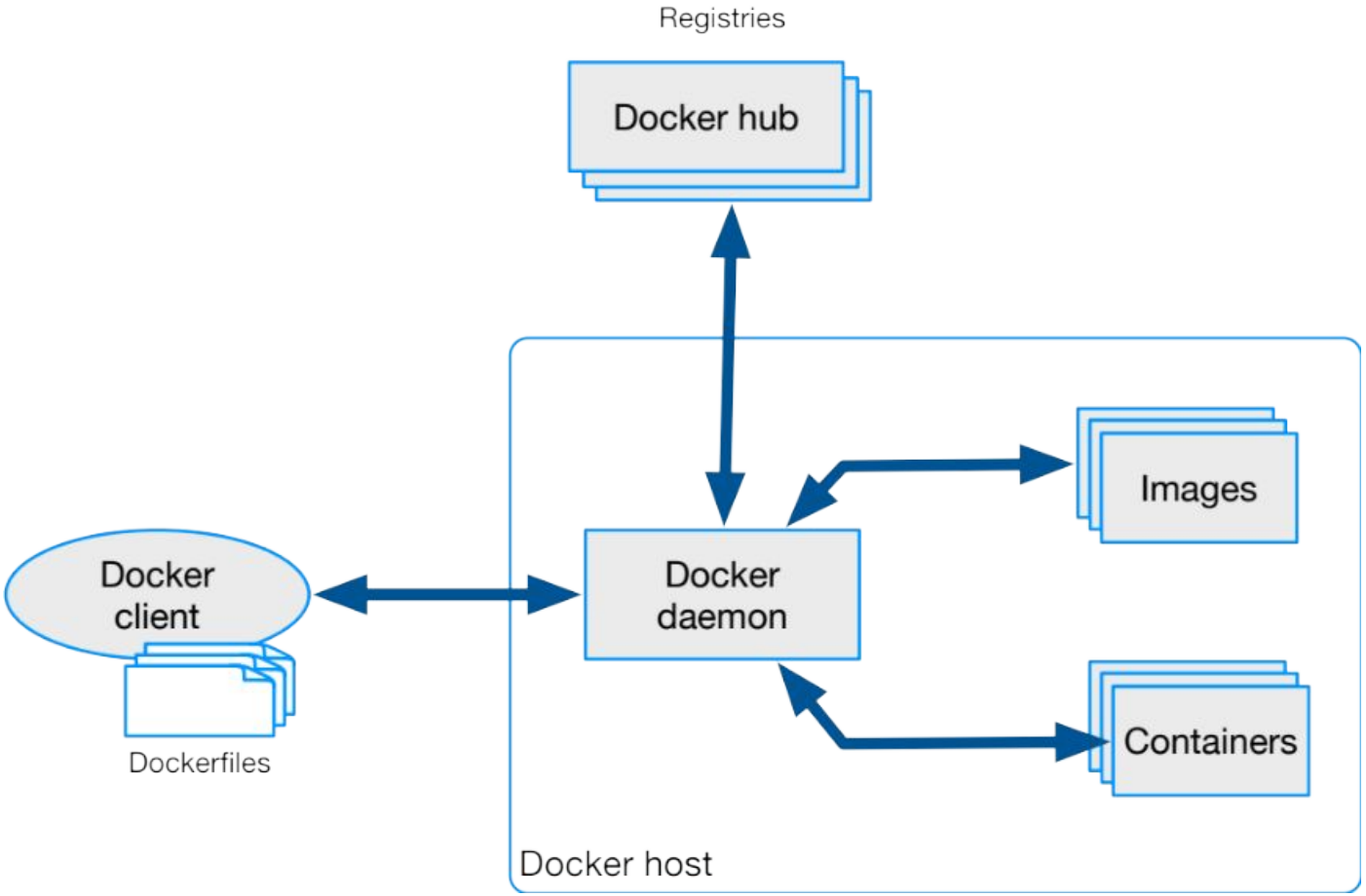
Contenedores vs Máquinas Virtuales

Caracterísitca	VM	Contenedor
Kernel	Completo	Compartido
Peso	Alto	Bajo
Inicio	Lento	Rápido
Aislamiento	Fuerte	Medio

Contenedores vs Máquinas Virtuales



Docker Containers



Beneficios de Contenedores

- Resuelven dependencias
  - Empaquetamiento y distribución de librerías, etc.
- Portabilidad entre Clouds
  - Que soporten Docker engine
- Implementación eficiente de Servicios
  - Buen soporte para SOA.
- Simplifican adopción de DevOps
  - Mismo equipo responsable del Desarrollo y Operación

Docker

- Docker permite:
  - Crear imágenes
  - Ejecutar contenedores
  - Distribuir aplicaciones
- Imagen:
  - Plantilla
- Contenedor:
  - Instancia en ejecución

Docker

- Dockerfile: Define cómo construir una imagen:
  - Base image
  - Dependencias
  - Código
  - Comando de ejecución

Ejemplo de Dockerfile

```
FROM maven:3.9.6-eclipse-temurin-17
WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn clean package

CMD ["mvn", "exec:java", "-Dexec.mainClass=com.test.Publisher"]
```

Docker

- Buenas prácticas Docker
  - Imágenes pequeñas
  - Multi-stage builds
  - Versionado explícito
- Ventajas de contenerización en pruebas
  - Entornos idénticos
  - Tests reproducibles
  - Eliminación de “works on my machine”
- Tipos de testing con contenedores
  - Unit tests
  - Integration tests
  - E2E tests
- Todos pueden ejecutarse en contenedores.

CI/CD

- Code → Build → Test → Image → Deploy
- La Integración Continua es una práctica de desarrollo donde los cambios de código se integran frecuentemente en un repositorio compartido, y cada integración dispara un proceso automático de verificación.
- Características clave:
  - Builds automáticos
  - Ejecución de tests unitarios e integración
  - Validación temprana de errores
  - Reducción de conflictos entre ramas
- Herramienta común:
  - GitHub Actions
  - Jenkins
- Beneficio principal:
  - Detectar errores lo antes posible en el ciclo de desarrollo.

CI/CD

- Entrega Continua (CD) extiende CI automatizando el despliegue del software en distintos entornos.
- Diferencia clave:
  - CI → valida el software
  - CD → lo despliega automáticamente
- Etapas típicas:
  - Build
  - Test
  - Staging
  - Producción
- Objetivo: Reducir el tiempo entre desarrollo y disponibilidad en producción.

Despliegue Automatizado

- El despliegue automatizado elimina la intervención manual en la puesta en producción del software.
- El objetivo es convertir el despliegue en un proceso repetible, confiable y automático.

- Problemas del despliegue manual:
  - Errores humanos frecuentes
  - Inconsistencias entre entornos
  - Deploys lentos y riesgosos
  - Dificultad para reproducir fallos

Contenerización como base del despliegue moderno

- La contenerización permite empaquetar aplicaciones junto con:
  - Dependencias
  - Configuración base
  - Entorno de ejecución
- Esto permite: *“Build once, run anywhere”*

Docker como estándar

- Docker permite:
  - Construcción de contenedores
  - Ejecución de imagenes
  - Distribución de aplicaciones
  - Reproducibilidad de entornos
- Imagen vs Contenedor
  Imagen:
  - Artefacto inmutable
  - Plantilla del sistema

  Contenedor:
  - Instancia en ejecución
  - Estado dinámico
- Analogía:
  - Imagen = clase
  - Contenedor = objeto

Docker como estándar

- Un Dockerfile define el proceso de construcción de una imagen.

- Incluye:
  - Imagen base
  - Copia de código
  - Instalación de dependencias
  - Comando de ejecución

Problema clásico resuelto por contenedores

- “Funciona en mi máquina”
- Causa:
  - Dependencias locales distintas
  - Versiones incompatibles
  - Configuración manual
- Solución:
  - Contenedores reproducibles.
- CI/CD con contenedores: Pipeline moderno:
  - Commit → Build Image → Test Container → Push Registry → Deploy

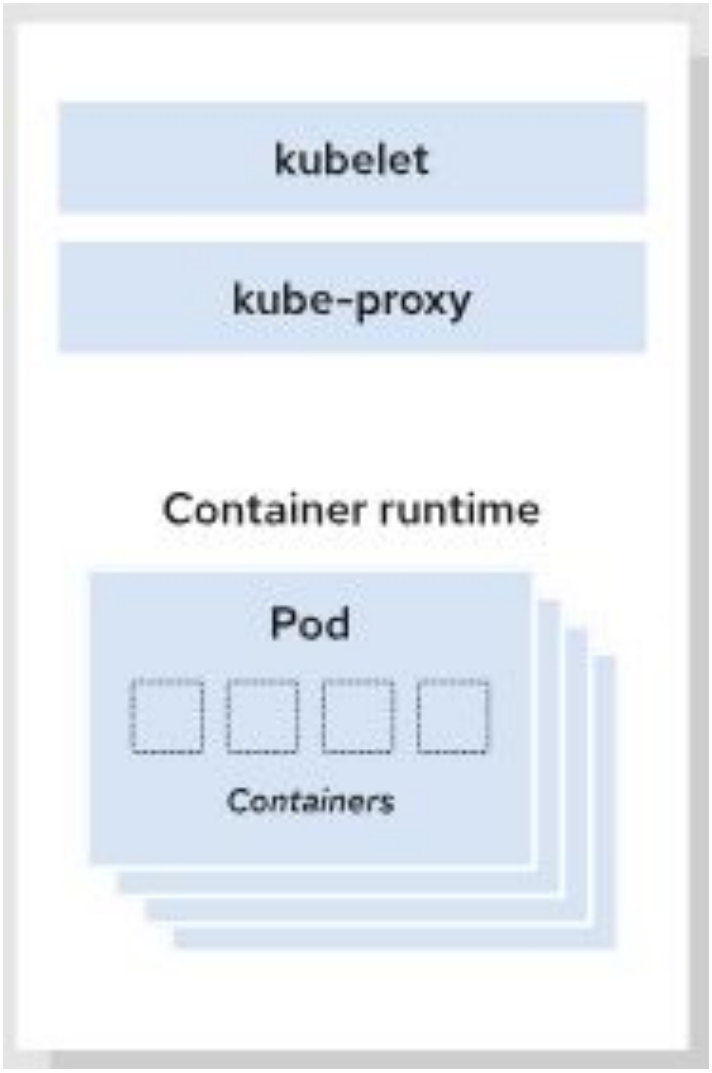
Orquestación de contenedores

- Cuando hay muchos contenedores, surge la necesidad de:
  - Coordinación
  - Escalado
  - Balanceo
  - Auto-recuperación

Kubernetes como orquestador

- Kubernetes permite:
  - Ejecutar contenedores a escala
  - Gestionar fallos automáticamente
  - Escalar aplicaciones dinámicamente
  - Desplegar sin downtime

Kubernetes como orquestador



Kubernetes como orquestador

- Pods
- Un Pod es la unidad mínima de ejecución en Kubernetes.
- Características:
  - Uno o más contenedores
  - Comparten red
  - Comparten almacenamiento
  - Puede contener uno o más contenedores
  - Comparten almacenamiento (volúmenes)
  - Se diseñan como unidad lógica de despliegue
  - Idea clave:
    Kubernetes no gestiona contenedores individuales, sino Pods.

Kubernetes como orquestador

- Ciclo de vida de un Pod

- Un Pod atraviesa estados:

  - Pending
  - Running
  - Succeeded
  - Failed
  - Unknown

Kubernetes como orquestador

- Un Worker Node es un nodo de trabajo que ejecuta aplicaciones reales.

- Componentes principales:
  - kubelet
  - kube-proxy
  - container runtime (Docker/containerd)

Kubernetes como orquestador

- kubelet (componente clave) es el agente principal del nodo.

- Funciones:
  - Recibe instrucciones del Control Plane
  - Crea y monitorea Pods
  - Reporta estado del nodo
  - Asegura que los contenedores estén corriendo

- kubelet es el “ejecutor local” de Kubernetes.

Kubernetes como orquestador

- El kubelet:
  - No decide qué ejecutar
  - Solo ejecuta lo que el Control Plane ordena
  - Monitorea salud de contenedores

- Ejemplo:
  - Si un contenedor falla → kubelet lo reinicia.

Kubernetes como orquestador

- kube-proxy (componente de red) maneja la red dentro del cluster.

- Funciones:
  - Routing de tráfico entre Pods
  - Implementación de Services
  - Load balancing básico
  - Manejo de reglas de red
- Basicamente:
  - Kubelet ejecuta Pods
  - Kube-proxy gestiona red del cluster

Kubernetes on-premises

- Implementación en infraestructura propia.
- Ventajas:
  - Control total del hardware
  - Cumplimiento regulatorio
  - Seguridad interna
- Desventajas:
  - Alta complejidad operativa
  - Costos de mantenimiento
  - Escalabilidad limitada física

Cloud

- Amazon EKS
- Google Kubernetes Engine
- Azure Kubernetes Service

- Ventajas:
  - Infraestructura gestionada
  - Escalabilidad automática
  - Menor carga operativa

Puntos Claves

- Conclusiones
  - kubelet ejecuta contenedores en nodos
  - kube-proxy maneja networking interno
  - Kubernetes automatiza operación de sistemas distribuidos

- La contenerización ha sido de ayuda en el proceso de CI/CD